第六章 与学习相关的技巧

本章的核心目标是：

- 理解不同参数更新方法之间的差异
- 掌握 SGD、Momentum、AdaGrad、Adam 的基本思想
- 明白权重初始化为何重要
- 理解 Xavier 和 He 初始化
- 掌握 Batch Normalization 的作用
- 理解过拟合、权值衰减与 Dropout
- 对超参数搜索建立基本直觉

6.1 参数更新

在前面的章节中，我们已经知道，训练神经网络的本质是：

根据损失函数对参数的梯度，反复更新参数，使损失变小。

最基础的更新方法就是：

$$W = W - \eta \frac{\partial L}{\partial W}$$

其中：

- W 表示参数
- η 表示学习率
- L 表示损失函数

但在实际训练中，仅仅“会更新”还不够，更关键的问题是：

怎样更新得更快、更稳定、更容易收敛？

这就引出了各种优化方法。

6.2 SGD

最基本的方法是随机梯度下降法，也就是 SGD。
更新规则：

$$W = W - \eta \frac{\partial L}{\partial W}$$

代码实现：

```
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def update(self, params, grads):
        for key in params.keys():
            params[key] -= self.lr * grads[key]
```

SGD 的优点：

- 简单
- 实现容易
- 思想直观

但它的问题也很明显：

- 容易震荡
- 收敛可能较慢
- 对学习率较敏感

6.3 Momentum

Momentum 的思想是：

> 更新时不仅看当前梯度，还参考过去的“速度”。

更新规则为：

$$v = \alpha v - \eta \frac{\partial L}{\partial W}$$

$$W = W + v$$

其中：

- v 表示速度
- α 是动量系数

代码实现：

```
class Momentum:
    def __init__(self, lr=0.01, momentum=0.9):
        self.lr = lr
        self.momentum = momentum
        self.v = None

    def update(self, params, grads):
        if self.v is None:
            self.v = {}
            for key, val in params.items():
```

```
                self.v[key] = 0

        for key in params.keys():
            self.v[key] = self.momentum * self.v[key] - self.lr * grads[key]
            params[key] += self.v[key]
```

Momentum 的优点在于：

- 能减轻震荡
- 通常比 SGD 更平滑
- 在某些方向上下降更快

6.4 AdaGrad

AdaGrad 的核心思想是：

> 为每个参数单独调整学习率。

具体来说，它会累计历史梯度平方和：

$$h = h + \frac{\partial L}{\partial W} \odot \frac{\partial L}{\partial W}$$

然后用：

$$W = W - \eta \frac{1}{\sqrt{h} + \epsilon} \frac{\partial L}{\partial W}$$

代码实现：

```
import numpy as np

class AdaGrad:
    def __init__(self, lr=0.01):
        self.lr = lr
        self.h = None

    def update(self, params, grads):
        if self.h is None:
            self.h = {}
            for key, val in params.items():
                self.h[key] = np.zeros_like(val)

        for key in params.keys():
            self.h[key] += grads[key] * grads[key]
            params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + 1e-7)
```

AdaGrad 的优点：

- 学习率可以自适应
- 在某些稀疏问题上表现很好

缺点是：

- 累积项会不断变大
- 后期学习率可能变得过小

6.5 Adam

Adam 可以理解为 Momentum 和 AdaGrad 的结合。
它同时维护：

- 梯度的一阶矩估计
- 梯度平方的二阶矩估计

因此它往往是一个很稳妥的优化器。
代码实现：

```
import numpy as np

class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.iter = 0
        self.m = None
        self.v = None

    def update(self, params, grads):
        if self.m is None:
            self.m, self.v = {}, {}
            for key, val in params.items():
                self.m[key] = np.zeros_like(val)
                self.v[key] = np.zeros_like(val)

        self.iter += 1
        lr_t = self.lr * (np.sqrt(1.0 - self.beta2 ** self.iter) / (1.0 - self.beta1 **
    self.iter))

        for key in params.keys():
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) * grads[key]
            self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) * (grads[key] ** 2)
            params[key] -= lr_t * self.m[key] / (np.sqrt(self.v[key]) + 1e-7)
```

Adam 常常具有以下优点：

- 收敛快
- 对学习率不那么敏感
- 在很多任务上表现稳定

6.6 权重初始值

除了优化器，另一个非常重要的问题是：

参数一开始该怎么初始化?

6.6.1 为什么不能初始化为 0

如果同一层所有权重都初始化为 0，那么:

- 所有神经元会学到同样的东西
- 神经元之间失去差异性
- 网络表达能力大幅下降

因此，权重必须随机初始化。

6.6.2 初始化过大或过小的问题

如果初始化值太大:

- 信号可能迅速放大
- 某些激活函数会进入饱和区
- 梯度变小，训练困难

如果初始化值太小:

- 信号会逐层衰减
- 学习速度变慢

因此，合理初始化的目标是:

> 让信号既不爆炸，也不消失。

6.6.3 Xavier 初始化

对于 sigmoid 和 tanh，常用 Xavier 初始化。
其思想是:

> 让各层之间信号的尺度尽量保持稳定。

若前一层节点数为 n，则可取:

$$W \sim \mathcal{N}\left(0, \frac{1}{n}\right)$$

代码形式:

```
W = np.random.randn(node_num, node_num) * np.sqrt(1.0 / node_num)
```

6.6.4 He 初始化

对于 ReLU，常用 He 初始化：

$$W \sim \mathcal{N}\left(0, \frac{2}{n}\right)$$

代码形式：

```
W = np.random.randn(node_num, node_num) * np.sqrt(2.0 / node_num)
```

可以简单记住：

- sigmoid / tanh 常配 Xavier
- ReLU 常配 He

6.7 Batch Normalization

随着网络变深，训练会变得困难。一个重要原因是：

> 每一层输入数据的分布在训练过程中不断变化。

Batch Normalization 的思想是：

> 对每一层输入做标准化，让分布更加稳定。

基本步骤：

1. 求 mini-batch 的均值
2. 求方差
3. 做标准化
4. 再进行缩放和平移

BatchNorm 的优点：

- 训练更稳定
- 学习更快
- 对初始化不那么敏感

6.8 过拟合

训练中常见的问题是：

> 训练集表现很好，但测试集表现变差。

这就是过拟合。
过拟合常见于：

- 数据太少
- 模型太复杂
- 训练时间太长

说明模型记住了训练样本的细节，而没有学到真正可泛化的规律。

6.9 权值衰减

为了缓解过拟合，可以在损失函数中加入参数过大的惩罚项：

$$L = L_0 + \frac{\lambda}{2} \sum W^2$$

这里：

- L_0 是原始损失
- λ 是正则化强度

这样做的效果是：

- 抑制过大的权重
- 让模型更简单
- 提高泛化能力

这通常也叫 L2 正则化。

6.10 Dropout

另一个经典的防过拟合方法是 Dropout。
它的思想是：

> 训练时随机屏蔽一部分神经元。

这样可以防止神经元之间形成过强依赖。
代码实现示意：

```
import numpy as np

class Dropout:
    def __init__(self, dropout_ratio=0.5):
        self.dropout_ratio = dropout_ratio
        self.mask = None

    def forward(self, x, train_flg=True):
        if train_flg:
            self.mask = np.random.rand(*x.shape) > self.dropout_ratio
            return x * self.mask
        else:
            return x * (1.0 - self.dropout_ratio)
```

```
    def backward(self, dout):
        return dout * self.mask
```

Dropout 的意义在于：

- 增加训练随机性
- 防止神经元之间过拟合
- 提高泛化能力

6.11 超参数验证

网络训练中，还有很多量不是训练出来的，而是人为设定的，例如：

- 学习率
- batch size
- dropout ratio
- 权值衰减系数

这些叫做：

超参数（hyperparameters）

超参数选择不当，会明显影响训练效果。

6.11.1 验证集

为了调节超参数，通常把数据分为：

- 训练集
- 验证集
- 测试集

其作用分别是：

- 训练集：用来学习参数
- 验证集：用来选择超参数
- 测试集：最终评估模型

6.11.2 超参数搜索

实际中常见做法是：

1. 先设定一个大范围
2. 在范围中随机采样若干组参数
3. 用验证集评估

4. 保留效果好的组合
5. 继续缩小范围搜索

这通常比一开始就做精细穷举更高效。

6.12 本章小结

第六章讨论的是：

不仅要会训练网络，还要学会把网络训练得更好。

需要重点掌握的内容包括：

- 优化器：SGD、Momentum、AdaGrad、Adam
- 初始化：Xavier 与 He
- Batch Normalization
- 过拟合、权值衰减、Dropout
- 超参数调节思路

这些技巧并没有改变神经网络的基本结构，但它们会显著影响：

- 收敛速度
- 训练稳定性
- 泛化能力

因此，它们是深度学习实践中极其重要的一部分。

6.13 附：本章核心代码

```
import numpy as np

class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def update(self, params, grads):
        for key in params.keys():
            params[key] -= self.lr * grads[key]


class Momentum:
    def __init__(self, lr=0.01, momentum=0.9):
        self.lr = lr
        self.momentum = momentum
        self.v = None

    def update(self, params, grads):
        if self.v is None:
```

```
            self.v = {}
            for key, val in params.items():
                self.v[key] = np.zeros_like(val)

        for key in params.keys():
            self.v[key] = self.momentum * self.v[key] - self.lr * grads[key]
            params[key] += self.v[key]


class AdaGrad:
    def __init__(self, lr=0.01):
        self.lr = lr
        self.h = None

    def update(self, params, grads):
        if self.h is None:
            self.h = {}
            for key, val in params.items():
                self.h[key] = np.zeros_like(val)

        for key in params.keys():
            self.h[key] += grads[key] * grads[key]
            params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + 1e-7)


class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.iter = 0
        self.m = None
        self.v = None

    def update(self, params, grads):
        if self.m is None:
            self.m, self.v = {}, {}
            for key, val in params.items():
                self.m[key] = np.zeros_like(val)
                self.v[key] = np.zeros_like(val)

        self.iter += 1
        lr_t = self.lr * (np.sqrt(1.0 - self.beta2 ** self.iter) / (1.0 - self.beta1 **
    self.iter))

        for key in params.keys():
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) * grads[key]
            self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) * (grads[key] ** 2)
            params[key] -= lr_t * self.m[key] / (np.sqrt(self.v[key]) + 1e-7)
```